# E-commerce Platform

Architecture & build plan — Go and PHP microservices, MongoDB, React

| Stack | Role |
| --- | --- |
| **React** (TypeScript, Vite) | Customer storefront and staff admin panel |
| **Go** | Auth, cart/orders/inventory, payments — the high-traffic, money-handling paths |
| **PHP** (Laravel) | Catalog, reviews and content — the editorial, CRUD-heavy side |
| **MongoDB** | Data store, one database per service, replica set for transactions |
| **RabbitMQ · Nginx · Docker** | Events between services, single entry point, local dev and deployment |

Prepared Nov 8, 2026 · Draft v1 for review. Section 12 lists the decisions to make before building — mark it up with what you want and the plan gets updated before any code is written.

# 1. What we're building

An online store where customers browse products, add them to a cart, check out and pay by card, and track their orders. Staff manage products, stock, orders, refunds and site content from an admin panel.

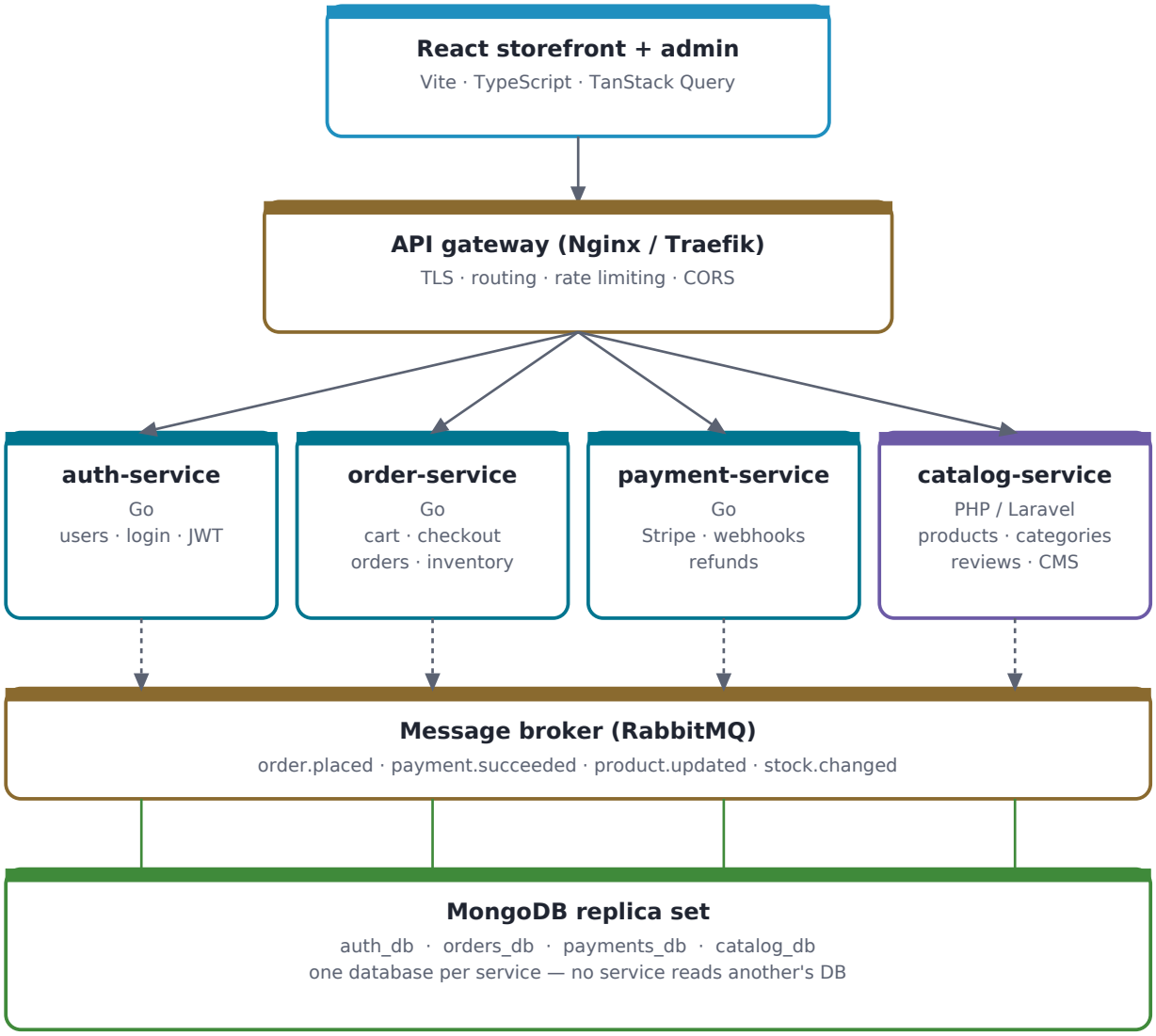
The backend is split into small services that each own one area of the business and their own data. Two languages are used deliberately, each where it fits best:

- **Go** handles the paths where correctness and concurrency matter most: logging in, reserving stock, placing orders and taking payment. It compiles to a single small binary and handles many simultaneous requests cheaply.
- **PHP with Laravel** handles the catalog and content: lots of forms, validation, image uploads and admin CRUD, which Laravel makes fast to build. The `mongodb/laravel-mongodb` package gives Eloquent models backed by MongoDB.
- **React** is one front-end codebase with two areas — the public storefront and a protected `/admin` section — talking to every service through one gateway URL.

## Core features (v1)

| Customers | Staff / admin |
|---|---|
| Browse and search products, filter by category, price, rating | Create and edit products, variants, images and categories |
| Product pages with variants (size, colour), reviews | Manage stock levels and low-stock alerts |
| Cart that survives logout and works for guests | View, fulfil, cancel and refund orders |
| Checkout with shipping address, tax and card payment | Moderate reviews, edit CMS pages and banners |
| Account: order history, addresses, password reset | Basic sales dashboard (revenue, orders, top products) |

# 2. System architecture




How a request flows: the browser only ever talks to the gateway. The gateway looks at the URL path and forwards the request to the service that owns it. Services never call each other's databases; when one service needs to react to something another did (an order was paid, a product's price changed), it listens for an event on the message broker.

### Why these pieces

| Component | Why it's there |
|---|---|
| API gateway | One domain for the front end (no CORS juggling), TLS termination, per-IP rate limits, routing by path. Nginx is simplest; Traefik auto-discovers Docker services. |
| RabbitMQ | Lets services stay decoupled. Mature, reliable clients exist for both Go (`amqp091-go`) and PHP (`php-amqplib`). NATS is a lighter alternative. |
| Database per service | Each service can change its schema without breaking others, and a slow query in one area can't lock up another. All four DBs can live on one MongoDB replica set to start. |
| Replica set | MongoDB multi-document transactions (needed for 'reserve stock + create order' atomically) require a replica set, even a single-node one in development. |
| Redis (optional) | Cache hot catalog reads and rate-limit counters. Not required for v1. |

# 3. The services in detail

## auth-service (Go)

- Registration, login, logout, email verification, password reset.
- Issues a short-lived **access token** (JWT, ~15 min) and a long-lived **refresh token** stored in an httpOnly, Secure cookie and in the database so it can be revoked.
- Signs JWTs with an **RS256 private key**; every other service verifies with the public key, so no service needs to call auth-service on each request.
- Roles: `customer`, `staff`, `admin`. Passwords hashed with Argon2id or bcrypt.

## order-service (Go)

- **Cart:** per user or per guest session ID; merged into the user's cart at login. Carts expire via a MongoDB TTL index.
- **Inventory:** stock per SKU. Stock is *reserved* when checkout starts and released if payment doesn't complete within ~15 minutes.
- **Orders:** snapshot of items, prices, tax, shipping and address at the moment of purchase, so later price changes never alter past orders. Status machine below.
- Keeps a local read-only copy of product name/price/SKU, updated from `product.updated` events, so checkout never depends on the PHP service being up.

```
pending_payment -> paid -> processing -> shipped -> delivered
       |            |
       v            v
   cancelled     refunded (full / partial)
```

### payment-service (Go)

- Creates a Stripe PaymentIntent (or equivalent) for an order and returns the client secret to React.
- Card details go straight from the browser to Stripe — they never touch our servers, which keeps PCI scope minimal.
- Receives Stripe webhooks, verifies the signature, records the payment, and publishes `payment.succeeded` or `payment.failed`. Handles refunds.
- Every webhook is processed idempotently (event ID stored) because providers retry.

### catalog-service (PHP / Laravel)

- Products with variants, categories (tree), brands, images, attributes.
- Search and filtering: MongoDB text indexes for v1; Atlas Search or Meilisearch if you need typo tolerance and facets later.
- Reviews and ratings (only verified buyers — listens to `order.delivered`).
- CMS: home-page banners, static pages (About, Shipping, Returns), promo blocks.
- Publishes `product.updated` / `product.deleted` events. Image uploads to S3-compatible storage with resizing via a queue job.

## 4. Data models (MongoDB)

MongoDB stores JSON-like documents, which suits products well: a shirt and a laptop can have different attributes in the same collection. Prices are stored as **integers in cents** to avoid rounding errors.

### catalog_db.products

```
{
  _id: ObjectId,
  slug: "classic-cotton-tee",
  name: "Classic Cotton Tee",
  description: "...",
  categoryIds: [ObjectId],
  brand: "Northline",
  images: [{ url, alt, position }],
  attributes: { material: "cotton", fit: "regular" },
  variants: [
    { sku: "TEE-BLK-M", options: { colour: "Black", size: "M" },
      priceCents: 2499, compareAtCents: 2999 }
  ],
  rating: { average: 4.6, count: 128 },
  status: "active",            // draft | active | archived
  createdAt, updatedAt
}
```

## orders_db.orders

```
{
  _id: ObjectId,
  number: "ORD-2026-000123",
  userId: ObjectId | null,      // null for guest checkout
  email: "buyer@example.com",
  items: [{ sku, productId, name, options, unitPriceCents, qty, lineTotalCents }],
  subtotalCents, discountCents, shippingCents, taxCents, totalCents,
  currency: "CAD",
  shippingAddress: { name, line1, line2, city, province, postalCode, country },
  status: "paid",
  history: [{ status: "pending_payment", at }, { status: "paid", at }],
  paymentId: ObjectId,
  createdAt, updatedAt
}
```

## Other collections

| Database | Collection | Key fields / indexes |
|---|---|---|
| auth_db | users | email (unique), passwordHash, roles[], addresses[], emailVerifiedAt |
| auth_db | refresh_tokens | userId, tokenHash, expiresAt (TTL index), revokedAt |
| orders_db | carts | userId or guestId (unique), items[], updatedAt (TTL index, e.g. 30 days) |
| orders_db | inventory | sku (unique), onHand, reserved, lowStockThreshold |
| orders_db | reservations | orderId, sku, qty, expiresAt (TTL index releases stock) |
| orders_db | product_cache | sku, name, priceCents, status — copy fed by catalog events |
| payments_db | payments | orderId, provider, providerIntentId (unique), amountCents, status |
| payments_db | webhook_events | providerEventId (unique) — guarantees idempotency |
| catalog_db | categories | slug (unique), parentId, path[] for breadcrumbs |
| catalog_db | reviews | productId, userId, orderId, rating, body, status (pending/approved) |
| catalog_db | pages | slug (unique), title, blocks[], published |

# 5. API design

All services speak JSON over REST under `/api/v1`. The gateway routes by the first path segment. Errors share one shape: `{ "error": { "code", "message", "fields" } }`.

| Method & path | Service | Purpose |
|---|---|---|
| POST /auth/register · /auth/login | auth (Go) | Create account / sign in, returns access token + sets refresh cookie |
| POST /auth/refresh · /auth/logout | auth (Go) | Rotate tokens / revoke session |
| GET · PATCH /me | auth (Go) | Profile and saved addresses |
| GET /catalog/products | catalog (PHP) | List with ?q=&category=&minPrice=&sort=&page= |
| GET /catalog/products/{slug} | catalog (PHP) | Product detail with variants |
| GET /catalog/categories | catalog (PHP) | Category tree for navigation |
| GET · POST /catalog/products/{id}/reviews | catalog (PHP) | Read reviews / post one (verified buyers) |
| GET /content/pages/{slug} | catalog (PHP) | CMS page content |
| GET · POST · PATCH · DELETE /cart/items | order (Go) | View cart, add, change qty, remove |
| POST /checkout | order (Go) | Validate cart, reserve stock, create pending order, return totals |
| GET /orders · /orders/{id} | order (Go) | Customer order history and detail |
| POST /payments/intent | payment (Go) | Create payment intent for an order, return client secret |
| POST /payments/webhook | payment (Go) | Provider callback (signature-verified, no JWT) |
| /admin/catalog/* | catalog (PHP) | Product, category, review, page management (staff role) |
| /admin/orders/* · /admin/inventory/* | order (Go) | Fulfil, cancel, adjust stock (staff role) |
| POST /admin/payments/{id}/refund | payment (Go) | Issue full or partial refund (admin role) |

### Checkout sequence

```
1. React   POST /checkout              -> order-service
           validates cart against product_cache, reserves stock (transaction),
           creates order {status: pending_payment}, returns orderId + totals
2. React   POST /payments/intent       -> payment-service
           creates provider PaymentIntent, returns clientSecret
3. React   confirms card with provider SDK (card data never hits our servers)
4. Provider POST /payments/webhook     -> payment-service
           verifies signature, stores payment, publishes payment.succeeded
5. order-service consumes payment.succeeded
           order -> paid, reservation -> committed, publishes order.placed
6. Notification worker sends confirmation email; React polls / shows success page
   If no payment within 15 min: reservation TTL expires, stock released, order cancelled
```

# 6. Events between services

| Event | Published by | Consumed by | Effect |
|---|---|---|---|
| product.updated / .deleted | catalog (PHP) | order | Refresh product_cache (name, price, status) |
| stock.changed | order | catalog | Show in-stock / out-of-stock on product pages |
| order.placed | order | notification, catalog | Confirmation email; mark buyer eligible to review |
| payment.succeeded / .failed | payment | order | Move order to paid, or release reservation |
| order.shipped / .delivered | order | notification, catalog | Shipping email; unlock review |
| refund.issued | payment | order, notification | Order to refunded, restock if returned |

Each event carries an `eventId`, `type`, `occurredAt` and `data`. Consumers record processed IDs so a redelivered message is harmless. Services write the event to an **outbox** collection in the same transaction as their data change, and a small relay publishes it — so a crash can never save the order but lose the event.

# 7. React front end

| Concern | Choice | Why |
|---|---|---|
| Build tool | Vite + TypeScript | Fast dev server, typed API contracts |
| Routing | React Router | Storefront and /admin in one app, lazy-loaded admin bundle |
| Server data | TanStack Query | Caching, retries, background refresh of product/order data |
| Client state | Zustand | Small store for cart drawer, UI flags, auth status |
| Styling | Tailwind CSS + a headless component kit | Quick, consistent UI without a heavy framework |
| Forms | React Hook Form + Zod | Validation shared with typed API schemas |
| Payments | Provider's React SDK (e.g. Stripe Elements) | Card fields hosted by the provider (PCI) |
| SEO (optional) | Next.js instead of Vite, or prerender product pages | Search engines index product pages better with server rendering |

## Pages

| Storefront | Admin (/admin, staff only) |
|---|---|
| Home · Category listing · Search results | Dashboard |
| Product detail | Products (list, create/edit, variants, images) |
| Cart · Checkout (address, shipping, payment) · Confirmation | Categories · Inventory |
| Login · Register · Forgot password | Orders (detail, fulfil, refund) |
| Account: orders, order detail, addresses, profile | Reviews moderation · Pages & banners · Users |

The access token lives in memory only (not localStorage); on page load React calls `/auth/refresh`, which uses the httpOnly cookie to get a new one. This protects tokens from XSS theft.

# 8. Repository layout

One monorepo keeps everything versioned together and lets one Docker Compose file start the whole system.

```
shop/
├─ docker-compose.yml          # all services + mongo + rabbitmq + gateway
├─ .env.example
├─ gateway/nginx.conf
├─ services/
│  ├─ auth-service/            # Go
│  │  ├─ cmd/server/main.go
│  │  ├─ internal/{http,domain,store,token}/
│  │  └─ Dockerfile
│  ├─ order-service/           # Go  (cart, checkout, orders, inventory)
│  │  ├─ cmd/{server,worker}/
│  │  └─ internal/{cart,checkout,order,inventory,events,store}/
│  ├─ payment-service/         # Go
│  │  └─ internal/{provider,webhook,refund,events}/
│  └─ catalog-service/         # PHP / Laravel
│     ├─ app/{Models,Http/Controllers,Jobs,Events}/
│     ├─ routes/api.php
│     └─ Dockerfile            # php-fpm + mongodb extension
├─ web/                        # React (Vite + TS)
│  └─ src/{api,features/{catalog,cart,checkout,account,admin},components,routes}/
├─ packages/contracts/         # OpenAPI specs + event JSON schemas (shared)
└─ deploy/                     # CI pipelines, k8s or compose-prod files
```

# 9. Key libraries

| Area | Go services | PHP service |
|---|---|---|
| HTTP | chi router (or Gin / Echo) | Laravel (routing, validation, middleware) |
| MongoDB | Official MongoDB Go Driver (v2) | PHP mongodb extension + mongodb/laravel-mongodb |
| JWT | golang-jwt/jwt (verify RS256) | firebase/php-jwt (verify RS256) |
| Messaging | rabbitmq/amqp091-go | php-amqplib (via a Laravel queue/consumer command) |
| Validation | go-playground/validator | Laravel Form Requests |
| Config & logs | env vars, slog (structured logs) | Laravel config, Monolog JSON |
| Testing | go test + testcontainers-go (real Mongo) | PHPUnit / Pest |
| Payments | stripe-go (or chosen provider) | — |

Exact versions are pinned when the project is scaffolded — whatever is current and stable at that point.

# 10. Security & reliability checklist

- HTTPS everywhere; HSTS at the gateway.
- JWT RS256 with short expiry; refresh tokens rotated on every use and revocable.
- Role checks in every service (don't trust the gateway alone).
- Input validation on every endpoint; MongoDB queries built from typed structs, never raw user JSON (prevents operator injection like `{"$gt": ""}`).
- Rate limits on login, register, password reset and checkout.
- Card data handled only by the payment provider; webhook signatures verified; idempotency on all payment and webhook handlers.
- Prices and totals always recalculated server-side at checkout — never trusted from the browser.
- Secrets in environment variables / a secret manager, never in git.
- Health endpoints (`/healthz`) on each service; structured logs with a request ID passed through the gateway for tracing.
- Daily MongoDB backups with a tested restore.

# 11. Build roadmap

| Phase | Deliverable | Done when |
|---|---|---|
| 0 · Foundation | Monorepo, Docker Compose (Mongo replica set, RabbitMQ, Nginx), CI, contracts folder | `docker compose up` starts every container with health checks green |
| 1 · Auth | auth-service + React login/register/account | Can register, log in, refresh, log out |
| 2 · Catalog | catalog-service + storefront listing/detail + admin product editor | Staff add products with images; customers browse and search |
| 3 · Cart & inventory | order-service cart, inventory, product_cache via events | Guest and user carts work and merge at login; stock shown correctly |
| 4 · Checkout & payment | Checkout flow, payment-service, webhooks, order status machine | Test-mode payment completes end to end; abandoned checkouts release stock |
| 5 · Admin operations | Order fulfilment, refunds, review moderation, CMS pages, dashboard | Staff can run the store day to day |
| 6 · Hardening | Emails, rate limits, logging, backups, load test, deploy to production | Launch checklist passes |

# 12. Decisions for you

These shape the build. Mark your choices (or add your own) and the plan will be updated before coding starts.

| # | Question | Default in this plan | Your choice |
|---|---|---|---|
| 1 | What do you sell? (physical goods, digital downloads, both) | Physical goods | |
| 2 | Single store or multi-vendor marketplace? | Single store | |
| 3 | Payment provider | Stripe (test mode first) | |
| 4 | Currency, tax and shipping regions | CAD; GST/HST/PST by province; Canada only | |
| 5 | Login options | Email + password; Google sign-in later | |
| 6 | Guest checkout allowed? | Yes | |
| 7 | Search quality needed | MongoDB text search for v1 | |
| 8 | SEO important for product pages? | Vite SPA (switch to Next.js if yes) | |
| 9 | Languages | English only | |
| 10 | Hosting | One VPS with Docker Compose; MongoDB Atlas | |
| 11 | Extra features (wishlist, coupons, loyalty points, subscriptions...) | Coupons only | |
| 12 | Expected traffic / catalog size | Small–medium (≤10k products) | |